

ISRM Lab 1: Vulnerability Assessment & Secure Remediation

Course: Information Security & Risk Management (ISRM)

Focus: OWASP Top 10, CWE Mapping, STRIDE Threat Modeling

LAB OBJECTIVE

The objective of this phase is to implement a **Student Management System** in two separate GitHub branches:

- **Vulnerable Branch** – Contains intentionally insecure code for vulnerability testing.
- **Fixed Branch** – Contains secure implementation with remediation applied.

This phase focuses only on:

1. Initial project setup
2. GitHub branch creation
3. Vulnerable implementation
4. Secure implementation

PART A: GITHUB REPOSITORY & BRANCH SETUP

1. Creating Git Repository

Commands Used:

```
git init

git add .

git commit -m "Initial_vulnerable_implementation"

git branch vulnerable-version

git checkout -b fixed-version
```

Branch Structure:

Branch Name	Purpose
vulnerable-version	Contains insecure implementation with OWASP vulnerabilities
fixed-version	Contains remediated and secure implementation

GitHub Repository Structure:

```
Student-Management-System/  
|  
|--- app.py  
|--- database.py  
|--- config.py  
|--- requirements.txt  
|--- templates/  
|--- static/  
|--- uploads/
```

PART B: VULNERABLE IMPLEMENTATION

1. SQL Injection Vulnerability (CWE-89)

Vulnerable Code:

```
def authenticate_user(username, password):  
  
    query = f"SELECT_*_FROM_users_WHERE  
    {username}='{username}''AND_password='{password}''"  
  
    cursor.execute(query)  
  
    return cursor.fetchone()
```

Issue:

- User input directly concatenated into SQL query
- Allows authentication bypass
- Enables database dumping

Attack Payload:

```
Username: admin' --  
Password: anything
```

2. Hardcoded Credentials (CWE-798)

Vulnerable Code:

```
SECRET_KEY = "super_secret_key"  
  
DB_PASSWORD = "admin123"
```

Issue:

- Credentials exposed in source code

- Easy privilege escalation
- Repository leak risk

3. Insecure Password Storage (CWE-256)

Vulnerable Code:

```
INSERT INTO users(username,password)
VALUES('admin','admin123')
```

Issue:

- Passwords stored in plain text
- Database compromise exposes all accounts

4. Broken Access Control (CWE-284)

Vulnerable Code:

```
@app.route('/admin')

def admin_panel():

    return render_template("admin.html")
```

Issue:

- No role verification
- Any user can access admin routes

5. File Upload Vulnerability (CWE-434)

Vulnerable Code:

```
filename = file.filename

file.save("uploads/" + filename)
```

Issue:

- No extension validation
- Allows malicious executable uploads

PART C: SECURE IMPLEMENTATION

1. Fixing SQL Injection

Secure Code:

```
def authenticate_user(username,password):  
  
    cursor.execute(  
        "SELECT_*_FROM_users WHERE_username=?  
        AND_password=?",  
        (username,password)  
    )  
  
    return cursor.fetchone()
```

Security Improvement:

- Parameterized queries used
- Prevents SQL injection attacks

2. Fixing Hardcoded Credentials

Secure Code:

```
import os  
  
SECRET_KEY = os.environ.get("SECRET_KEY")  
  
DB_PASSWORD = os.environ.get("DB_PASSWORD")
```

Security Improvement:

- Secrets moved to environment variables
- Prevents source code exposure

3. Fixing Password Storage

Secure Code:

```
from werkzeug.security import  
generate_password_hash  
  
hashed_password =  
generate_password_hash(password)
```

Security Improvement:

- Passwords stored as hashes
- Prevents credential leakage

4. Fixing Access Control

Secure Code:

```
@app.route('/admin')

@login_required

def admin_panel():

    if session['role'] != 'admin':
        return "Access_Denied"

    return render_template("admin.html")
```

Security Improvement:

- Role-Based Access Control implemented
- Unauthorized users blocked

5. Fixing File Upload Vulnerability

Secure Code:

```
from werkzeug.utils import secure_filename

filename = secure_filename(file.filename)

ALLOWED_EXTENSIONS = {'png', 'jpg', 'pdf'}

if extension in ALLOWED_EXTENSIONS:

    file.save(filepath)
```

Security Improvement:

- File extension validation added
- Filename sanitization implemented

PART D: GIT DIFFERENCE BETWEEN BRANCHES

Commands Used to Compare Changes:

```
git diff vulnerable-version fixed-version
```

Files Compared:

- app.py
- database.py
- config.py

PART E: IMPLEMENTATION SUMMARY

Vulnerability	CWE	Fix Applied
SQL Injection	CWE-89	Parameterized Queries
Hardcoded Credentials	CWE-798	Environment Variables
Plain Text Passwords	CWE-256	Password Hashing
Broken Access Control	CWE-284	RBAC Authentication
File Upload Vulnerability	CWE-434	Secure Filename Validation

CONCLUSION

In Phase 1 of the ISRM Lab, two separate GitHub branches were created to demonstrate both insecure and secure implementations of a Student Management System. The vulnerable branch intentionally contained multiple OWASP Top 10 vulnerabilities for educational analysis, while the fixed branch implemented secure coding practices including parameterized queries, RBAC, password hashing, secure file validation, and environment-based secret management.

This implementation phase establishes the foundation for further vulnerability assessment, STRIDE modeling, penetration testing, and remediation verification in later lab phases.

— End of Report —